CODE

```
// Full RPS game for Bela-Arduino
// Button start, 3 countdown LEDs, arm wiggle, 5 finger servos, 2 flex sens
ors,
// and 3 result LEDs (win/lose/tie).
//
// Uses pwmWrite(pin, duty, 50) with your map / duty scheme.

#include <libraries/BelaArduino/Arduino.h>
#include <libraries/BelaArduino/PdArduino.h>
#include <libraries/Watcher/Watcher.h>

// --- Pins ------------------------------------------------
const int buttonPin = 7;       // button (to GND) - use INPUT_PULLUP

// Countdown LEDs (1s rhythm)
const int ledCount1 = 8;
const int ledCount2 = 9;
const int ledCount3 = 11;

// Result LEDs
const int ledWin  = 12;
const int ledLose = 13;
const int ledTie  = 10;

// Servos: five fingers (you used these earlier)
const int servoFinger0 = 0;   // finger servo 1 (three-finger side)
const int servoFinger1 = 2;   // finger servo 2
const int servoFinger2 = 3;   // finger servo 3
const int servoFinger3 = 4;   // finger servo 4
const int servoFinger4 = 5;   // finger servo 5

// Arm rotation servo (wiggle)
const int servoArm = 1;
```

```
// Flex sensors (analog channels)
const int flexA = 0;   // analog 0
const int flexB = 1;   // analog 1

// --- Servo/PWM params (from your Task 2) ------------------
// 0° → 500us, 180° → 2500us, period 20ms (50Hz)
float minDuty = 0.025;   // 500us / 20000us
float maxDuty = 0.125;   // 2500us / 20000us

// --- Gesture angle definitions (tweak if mechanical limits) ---
const int FINGER_OPEN  = 30;   // open
const int FINGER_CLOSE = 160;  // closed (rock)
const int ARM_CENTER   = 90;
const int ARM_LEFT90   = 60;   // center - 30
const int ARM_RIGHT90  = 120;  // center + 30

// timing (ms)
const int t_wait_before_start = 1000;
const int t_count_step = 800;   // each countdown LED step duration
const int t_arm_step = 300;     // delay between arm angle steps (controls s
peed)
const int t_after_gesture = 1200;
const int t_result_show = 2500;

// --- Helpers ----------------------------------------------
// Write an angle (0..180) to a servo pin using your map→pwmWrite pattern
void writeServoAngle(int pin, int angle) {
  if (angle < 0) angle = 0;
  if (angle > 180) angle = 180;
  float duty = map(angle, 0, 180, (int)(minDuty * 1000), (int)(maxDuty * 100
0)) / 1000.0;
  pwmWrite(pin, duty, 50);
}

// Set robot hand pose by move code: 0=ROCK, 1=PAPER, 2=SCISSORS
void setRobotPose(int move) {
  if (move == 0) { // ROCK: all fingers closed
    writeServoAngle(servoFinger0, FINGER_CLOSE);
```

```
    writeServoAngle(servoFinger1, FINGER_CLOSE);
    writeServoAngle(servoFinger2, FINGER_CLOSE);
    writeServoAngle(servoFinger3, FINGER_CLOSE);
    writeServoAngle(servoFinger4, FINGER_CLOSE);
  } else if (move == 1) { // PAPER: all open
    writeServoAngle(servoFinger0, FINGER_OPEN);
    writeServoAngle(servoFinger1, FINGER_OPEN);
    writeServoAngle(servoFinger2, FINGER_OPEN);
    writeServoAngle(servoFinger3, FINGER_OPEN);
    writeServoAngle(servoFinger4, FINGER_OPEN);
  } else { // SCISSORS: index+middle open, others closed
    // Choose finger mapping: finger0 & finger1 as index+middle for example
    writeServoAngle(servoFinger0, FINGER_OPEN);   // index
    writeServoAngle(servoFinger1, FINGER_OPEN);   // middle
    writeServoAngle(servoFinger2, FINGER_CLOSE);  // ring
    writeServoAngle(servoFinger3, FINGER_CLOSE);  // little
    writeServoAngle(servoFinger4, FINGER_CLOSE);  // thumb (adjust if map
ping diff)
  }
}

// Read human move from two flex sensors: returns 0=ROCK,1=PAPER,2=S
CISSORS
int readHumanMove(float baselineA, float baselineB) {
  const int samples = 20;
  float sumA = 0.0;
  float sumB = 0.0;
  for (int i = 0; i < samples; ++i) {
    sumA += analogRead(flexA);
    sumB += analogRead(flexB);
    delay(5);
  }
  float vA = sumA / samples;
  float vB = sumB / samples;
  Serial.print("Flex readings: A=");
  Serial.print(vA);
  Serial.print(" B=");
  Serial.println(vB);
```

```
  const float THRESH = 0.05; // adjust if needed (analogRead returns 0..1)
  bool bentA = (vA > baselineA + THRESH);
  bool bentB = (vB > baselineB + THRESH);

  if (bentA && bentB) return 0;       // ROCK
  if (!bentA && !bentB) return 1;     // PAPER
  return 2;                           // SCISSORS (one bent)
}

// Simple decide winner: 0 draw, 1 human wins, -1 robot wins
int decideWinner(int human, int robot) {
  if (human == robot) return 0;
  if ((human == 0 && robot == 2) ||
      (human == 1 && robot == 0) ||
      (human == 2 && robot == 1)) return 1;
  return -1;
}

// Show result with result LEDs: light single LED for the result
void showResultLED(int result) {
  // Clear all
  digitalWrite(ledWin, LOW);
  digitalWrite(ledLose, LOW);
  digitalWrite(ledTie, LOW);

  if (result == 0) { // draw
    digitalWrite(ledTie, HIGH);
    Serial.println("Result: DRAW");
  } else if (result == 1) {
    digitalWrite(ledWin, HIGH);
    Serial.println("Result: HUMAN WINS");
  } else {
    digitalWrite(ledLose, HIGH);
    Serial.println("Result: ROBOT WINS");
  }
}
```

```
// Countdown rhythm using three leds (on/off one by one)
void countdownRhythm() {
  // All on then off sequentially as in your earlier snippet
  digitalWrite(ledCount1, HIGH);
  digitalWrite(ledCount2, HIGH);
  digitalWrite(ledCount3, HIGH);
  delay(t_count_step);

  digitalWrite(ledCount3, LOW);
  delay(t_count_step);

  digitalWrite(ledCount2, LOW);
  delay(t_count_step);

  digitalWrite(ledCount1, LOW);
  delay(t_count_step);
}

// Arm wiggle: center → +30 → center → -30 → center
// This function also triggers the robot to show a gesture at the same time
void armWiggleAndShowGesture(int robotMove) {
  // ensure arm center
  writeServoAngle(servoArm, ARM_CENTER);
  delay(150);

  // rotate right (+30)
  for (int a = ARM_CENTER; a <= ARM_RIGHT90; a += 5) {
    writeServoAngle(servoArm, a);
    delay(t_arm_step / 3);
  }

  // immediate back to center
  for (int a = ARM_RIGHT90; a >= ARM_CENTER; a -= 5) {
    writeServoAngle(servoArm, a);
    delay(t_arm_step / 3);
  }

  // rotate left (-30)
```

```
  for (int a = ARM_CENTER; a >= ARM_LEFT90; a -= 5) {
    writeServoAngle(servoArm, a);
    delay(t_arm_step / 3);
  }

  // back to center while robot shows gesture
  for (int a = ARM_LEFT90; a <= ARM_CENTER; a += 5) {
    writeServoAngle(servoArm, a);
    // set robot pose while returning to center (this is where robot reveal hap
pens)
    setRobotPose(robotMove);
    delay(t_arm_step / 3);
  }
}

// --- Setup ------------------------------------------------
void setup() {
  Serial.begin(115200);
  Serial.println("RPS Game starting...");

  // button
  pinMode(buttonPin, INPUT_PULLUP);

  // countdown leds
  pinMode(ledCount1, OUTPUT);
  pinMode(ledCount2, OUTPUT);
  pinMode(ledCount3, OUTPUT);

  // result leds
  pinMode(ledWin, OUTPUT);
  pinMode(ledLose, OUTPUT);
  pinMode(ledTie, OUTPUT);

  // servos
  pinMode(servoFinger0, OUTPUT);
  pinMode(servoFinger1, OUTPUT);
  pinMode(servoFinger2, OUTPUT);
  pinMode(servoFinger3, OUTPUT);
```

```
  pinMode(servoFinger4, OUTPUT);
  pinMode(servoArm, OUTPUT);

  // initial positions → paper
  setRobotPose(1);
  writeServoAngle(servoArm, ARM_CENTER);

  // seed random from analog noise
  float rv = analogRead(2); // use analog channel as seed (or flex)
  randomSeed((unsigned long)(rv * 100000.0f));

  Serial.println("Ready. Press button to play.");
}

// --- Main loop ----------------------------------------
void loop() {
  // Wait for button press (active low, INPUT_PULLUP)
  if (digitalRead(buttonPin) == LOW) {
    // Debounce / wait for release
    delay(30);
    if (digitalRead(buttonPin) != LOW) return; // spurious
    // Wait stable press
    while (digitalRead(buttonPin) == LOW) delay(5);

    Serial.println("Button pressed - starting round");
    delay(t_wait_before_start); // 1 second before game start

    // Countdown rhythm
    countdownRhythm();

    // choose robot move randomly: 0=rock,1=paper,2=scissors
    int robotMove = random(0, 3);
    Serial.print("Robot move (rand): ");
    Serial.println(robotMove);

    // Arm wiggle and reveal robot pose while returning to center
    armWiggleAndShowGesture(robotMove);
```

```
  delay(t_after_gesture);

  // Calibrate flex baseline quickly (ask user to hold open/paper)
  Serial.println("Calibrating flex sensors - hold your hand OPEN (paper) for
2s");
  float sumA = 0, sumB = 0;
  const int calSamples = 30;
  for (int i = 0; i < calSamples; ++i) {
    sumA += analogRead(flexA);
    sumB += analogRead(flexB);
    delay(30);
  }
  float baseA = sumA / calSamples;
  float baseB = sumB / calSamples;
  Serial.print("Baseline A: "); Serial.println(baseA);
  Serial.print("Baseline B: "); Serial.println(baseB);

  delay(300); // small pause to let human prepare to show

  // Read human gesture
  Serial.println("Reading human gesture...");
  int humanMove = readHumanMove(baseA, baseB);
  Serial.print("Human move: ");
  if (humanMove == 0) Serial.println("ROCK");
  else if (humanMove == 1) Serial.println("PAPER");
  else Serial.println("SCISSORS");

  // Decide result
  int result = decideWinner(humanMove, robotMove);

  // Show result LEDs
  showResultLED(result);

  // Also print robot/human mapping
  const char* names[3] = {"ROCK", "PAPER", "SCISSORS"};
  Serial.print("Robot: "); Serial.print(names[robotMove]);
  Serial.print(" | Human: "); Serial.println(names[humanMove]);
```

```
    delay(t_result_show);

    // Reset result leds and robot pose to neutral (paper)
    digitalWrite(ledWin, LOW);
    digitalWrite(ledLose, LOW);
    digitalWrite(ledTie, LOW);
    setRobotPose(1);
    writeServoAngle(servoArm, ARM_CENTER);

    Serial.println("Round complete. Ready for next round.");
  }

  // small idle delay
  delay(50);
}
```